

Informatik
Hochschule Luzern (Schweiz)

Architekturdokumentation

Skitourenverwaltung & 3x3 Sicherheitsmatrix (arc42 v8)

WEBLAB

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Colin Felber

von

Luzern (Schweiz)

Inhaltsverzeichnis

1	Einführung & Ziele	2
1.1	Qualitätsziele	2
1.2	Stakeholder	2
2	Kontext & Abgrenzung	3
2.1	Fachlicher Kontext	3
2.2	Technischer Kontext	3
2.3	In Scope	3
2.4	Out of Scope	4
3	Lösungsstrategie	5
4	Bausteinsicht	6
4.1	Bausteinsicht Level 1: Gesamtsystem Whitebox	6
4.2	Bausteinsicht Level 2: Frontend-Architektur	6
4.3	Bausteinsicht Level 2: Backend-Architektur	7
5	Laufzeitsicht	9
5.1	Szenario 1: Registrierung & Login	9
5.2	Szenario 2: Tour erstellen mit 3x3 Sicherheitsmatrix	9
5.3	Szenario 3: Tour-Beitritt & Notfallkontakt-Abfrage	10
6	Verteilungssicht & Deployment	11
6.1	Container & Multi-Stage Builds	11
6.2	Automatisierte CI/CD Pipeline (GitHub Actions)	11
7	Querschnittskonzepte	13
7.1	Ownership, Datenschutz & Sicherheit	13
7.2	Datenmodell & Konsistenz	14
7.3	Frontend-Konzepte & Reaktivität	14
7.4	Bewusste Architektur-Abweichungen & Trade-offs	15
7.5	Testing	15
8	Architekturentscheidungen (ADRs)	16
8.1	ADR-01: Backend mit NestJS (Modularer Monolith)	16
8.2	ADR-02: Frontend mit Angular 19 Standalone & Signals	16
8.3	ADR-03: Persistenz mit MongoDB & Mongoose (Embedded Aggregates)	16
8.4	ADR-04: Zustandsloses JWT mit Passport & bcrypt	16
8.5	ADR-05: Deployment mit Docker Compose & Nginx Reverse Proxy	17
8.6	ADR-06: Restriktiver Notfallkontakt-Zugriff (Least Privilege)	17
9	Qualitätsanforderungen	18
9.1	Lighthouse	19
10	Risiken & technische Schulden	20

1 Einführung & Ziele

Wer eine Skitour für eine Gruppe leitet, trägt viel Verantwortung. Kommt es zu einem Notfall am Berg, muss die Tourenleitung sofort die Notfallkontakte der Teilnehmenden griffbereit haben. Zudem fordert die Rechtsprechung im Ernstfall den Nachweis einer sorgfältigen Tourenplanung nach anerkannten Standards, insbesondere der 3x3-Sicherheitsmatrix (Verhältnisse, Gelände, Mensch).

Normale Chat-Apps oder Vereins-Tools reichen dafür nicht aus:

Notfallkontakte fehlen oder sind unauffindbar, und die 3x3-Planung kann nirgends strukturiert festgehalten werden. Die vorliegende Webapplikation schliesst diese Lücke.

1.1 Qualitätsziele

Prio	Qualitätsziel	Bedeutung für die Applikation
1	Datenschutz & Vertraulichkeit	Notfallkontakte sind hochsensibel. Ausschliesslich autorisierte Tourenleiter der jeweiligen Tour dürfen sie einsehen (Principle of Least Privilege).
2	Schnelligkeit & Usability	Schnelle Bedienung am Handy und Desktop. Die 3x3-Matrix ist in wenigen Minuten ausgefüllt, Notfallkontakte sind mit einem Klick sichtbar.
3	Wartbarkeit & Testbarkeit	Klare Entkopplung: Smart/Dumb-Muster im Frontend, modularer Monolith mit NestJS im Backend, abgesichert durch Unit- und E2E-Tests.
4	Ein-Befehl-Start	Vollständiger Start des Stacks via <code>docker compose up --build</code> ohne manuelle Vorabininstallation von Node oder Datenbanken.

1.2 Stakeholder

Rolle	Wer ist das?	Erwartungshaltung
Tourenleiter/in	Leiter/in von SAC- oder privaten Touren	Strukturierte 3x3-Dokumentation zur rechtlichen Absicherung, Teilnehmerübersicht und Notfallkontakte im Ernstfall.
Teilnehmende	Mitglieder der Tourengruppe	Einfacher Beitritt per Link, Übersicht zu Treffpunkt, Route und Ausrüstung, garantierter Schutz der Notfalldaten.
Dozent / Experte	Dominik Witschard (HSLU)	Klare Softwarearchitektur, nachvollziehbar begründete Technologiewahl, Testabdeckung und verständlicher Code.

2 Kontext & Abgrenzung

2.1 Fachlicher Kontext

Das System fokussiert sich auf die Planung, Durchführung und rechtliche Absicherung von Touren. Externe Daten (SLF-Lawinenbulletin, Wetterdaten) werden vom Tourenleiter manuell in die 3x3-Matrix übertragen. GPX-Routentracks können als Datei hochgeladen werden.

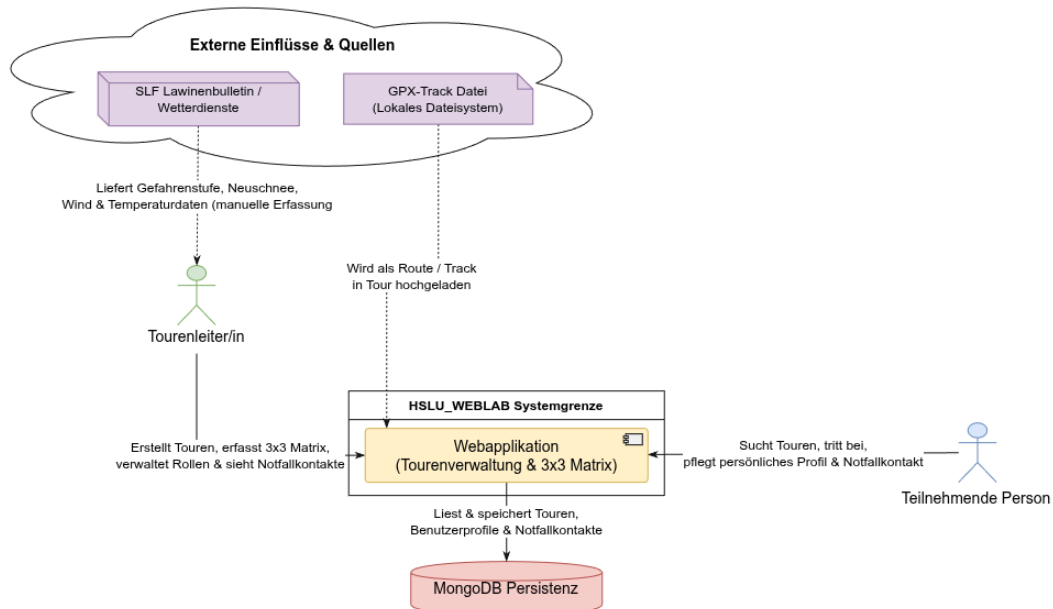


Figure 1: Systemkontext: Benutzer, Schnittstellen und Umsysteme (PlantUML: 01_kontext_abgrenzung.puml)

2.2 Technischer Kontext

- Der Browser kommuniziert über HTTP (Port 80) mit Nginx.
- Nginx leitet API-Aufrufe (/api/*) intern an das NestJS-Backend (Port 4566) weiter.
- Das Backend greift über das MongoDB-Wire-Protokoll (Port 27017) auf die Datenbank zu.

2.3 In Scope

- + **Touren-CRUD:** Vollständiges Erstellen, Anzeigen, Bearbeiten und Löschen von Touren (Mindestanforderung).
- + **Unterschiedliche Darstellungsformen:** Präsentation der Tourendaten in drei inhaltlich eigenständigen Darstellungsformen (Mindestanforderung): Kachel-/Katalogansicht (TourPreview), interaktive geografische Karte (Map mit Leaflet & GPX-Visualisierung) und strukturierte 3x3-Risikomatrix (SecurityMatrixDisplay).
- + **Digitale 3x3-Sicherheitsmatrix:** Strukturierte Risikobeurteilung nach Werner Munter (Verhältnisse, Gelände, Mensch).
- + **Notfallkontakt-Erfassung:** Hinterlegung bei der Registrierung und Bearbeitung im Profil.
- + **Geschützter Notfallkontakt-Zugriff:** Zugriff auf Notfalldaten ausschliesslich für autorisierte Tourenleiter der jeweiligen Tour (Principle of Least Privilege).
- + **Teilnehmer- und Rollenverwaltung:** Beitritt per Link und Verwaltung von Leiter- und Teilnehmerrollen.
- + **GPX-Routenupload:** Hochladen und Anzeigen von GPX-Tracks direkt in der Tour.

2.4 Out of Scope

- **Native Mobile Apps:** Keine eigenständigen Apps für iOS oder Android (reine responsive Web-App).
- **Automatisches Lawinenbulletin:** Kein API-Scraping von SLF- oder Wetterdiensten (manuelle Erfassung).
- **E-Mail-Dienste:** Keine E-Mail-Verifikation und kein Passwort-Reset per Mail.
- **Live-GPS-Tracking:** Keine Turn-by-Turn-Navigation oder Live-Ortung während der Tour.

3 Lösungsstrategie

Die Lösungsstrategie beschreibt die grundlegenden architektonischen Leitlinien und Ansätze, mit denen die in Kapitel 1 definierten Qualitätsziele (Datenschutz, Schnelligkeit/Usability, Wartbarkeit/Testbarkeit, 1-Befehl-Start) realisiert werden:

1. **Entkoppelter Fullstack-Webstack (Modularer Monolith REST API + Standalone SPA):**

Das Gesamtsystem wird in ein typisiertes TypeScript-Frontend (Angular 19) und ein strukturiertes TypeScript-Backend (NestJS 11) getrennt, die über eine RESTful JSON-Schnittstelle kommunizieren. Im Backend sichert der Aufbau als Modularer Monolith (auth, tours, users) eine saubere Domänentrennung und hohe Testbarkeit ohne die Netzwerk- und Betriebs-Komplexität von Microservices (-> ADR-01).

2. **Datenschutz & Least Privilege an der Servergrenze:** Hochsensible Notfallkontakte werden serverseitig in allen Standard-DB-Abfragen per Mongoose-Projektion maskiert (`.select(' - emergencyContact ')`). Eine Einsicht erfolgt ausschliesslich über einen gesonderten Endpunkt, der die Tourenleiter-Rolle des anfragenden JWT-Nutzers zwingend prüft. Sensible Daten werden niemals unbefugt an den Browser ausgeliefert (-> ADR-04, ADR-06).

3. **Dokumentenorientiertes Aggregat-Muster (MongoDB):** Die 3x3-Sicherheitsmatrix und Notfallkontakte bilden fachlich feste Aggregate mit ihrer jeweiligen Entität (Tour bzw. Person). Sie werden direkt als BSON-Subdokumente eingebettet. Dies verhindert teure Joins, sichert atomare Updates (`$addToSet`) und ermöglicht schemalose Felderweiterungen ohne SQL-Migrationen (-> ADR-03).

4. **Framework-native Reaktivität (Signals & Smart/Dumb):** Im Frontend wird auf schwere externe State-Libraries (wie NgRx/Redux) verzichtet. Stattdessen etablieren Angular Signals (`signal`, `computed`, `httpResource`) und das Smart/Dumb-Komponentenmuster eine feingranulare, transparente Reaktivität mit minimalem Boilerplate und automatischer Change Detection (`OnPush`) (-> ADR-02).

5. **Zero-CORS Reverse-Proxy & Container-Deployment:** Ein schlanker Nginx-Container fungiert als einziger öffentlicher Einstiegspunkt (Port 80). Er liefert das statische Angular-Produktionsbundle aus und leitet API-Aufrufe (`/api/*`) intern an das NestJS-Backend weiter. Dies eliminiert CORS-Probleme vollständig und ermöglicht den reproduzierbaren 1-Befehl-Start via Docker Compose (-> ADR-05).

4 Bausteinsicht

4.1 Bausteinsicht Level 1: Gesamtsystem Whitebox

Das System läuft vollständig isoliert in Docker Compose:

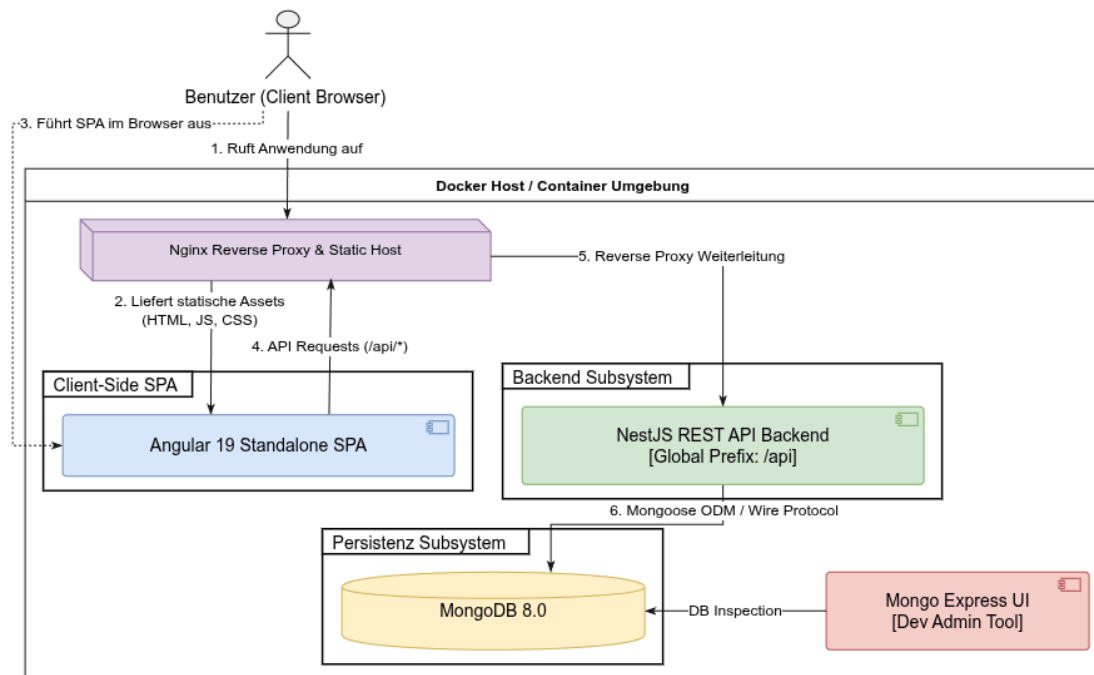


Figure 2: Bausteinsicht Level 1: System-Whitebox mit Containern und Ports (PlantUML: 02_bausteinsicht_level1.puml)

Container	Technologie	Verantwortung & Schnittstellen
HSLU_WEBLAB_frontend	nginx:alpine	Haupteinstiegspunkt, Liefert das Angular-Bundle aus und proxied /api/* an das Backend weiter.
backend	NestJS 11, Node.js 24	REST-API, Verarbeitet Business-Logik, Validierung und Berechtigungsprüfungen.
mongo	mongo:8	MongoDB-Server, Speichert Daten dauerhaft im Volume mongo-data.
mongo_express	mongo-express:1	Web-Admin-Tool, zur Datenbank-Inspektion während der Entwicklung.

4.2 Bausteinsicht Level 2: Frontend-Architektur

Das Frontend ist nach Features und dem Smart/Dumb-Muster strukturiert:

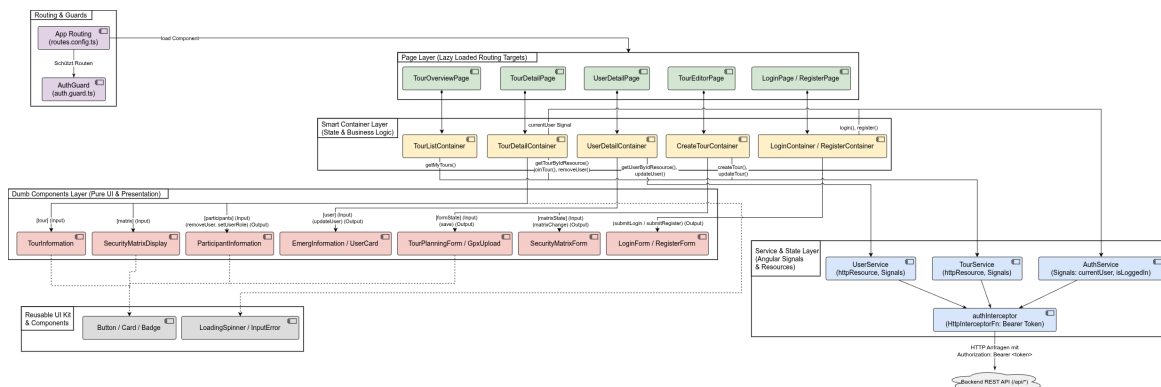


Figure 3: Bausteinsicht Level 2: Frontend-Aufbau mit Smart Containern und Dumb Components (PlantUML: 03_frontend_architektur.puml)

Ordner / Bereich	Inhalt & Verantwortung
features/tour-management/	Tourenverwaltung in verschiedenen Darstellungsformen: Katalogübersicht (TourPreview), interaktive Karte (Map mit Leaflet/GPX), Detailansicht (TourInformation), Teilnehmerliste und 3x3-Matrix (SecurityMatrixDisplay).
features/tour-editor/	Formulare zur Tourenerstellung und Bearbeitung der 3x3-Matrix, GPX-Upload (TourForm, SecurityMatrixForm).
features/user/	Benutzerprofil, Bearbeitung der Notfallkontakte, geschützte Notfallansicht (UserCard, EmergInformation).
features/auth/	Login- und Registrierungsformulare, AuthService, authInterceptor, authGuard.
components/	Wiederverwendbares UI-Kit (Button, Card, Badge, LoadingSpinner, InputFieldError).
pipes/	Wiederverwendbare Hilfs-Pipes (ShortenerPipe).

4.3 Bausteinsicht Level 2: Backend-Architektur

Das Backend ist als modularer Monolith organisiert:

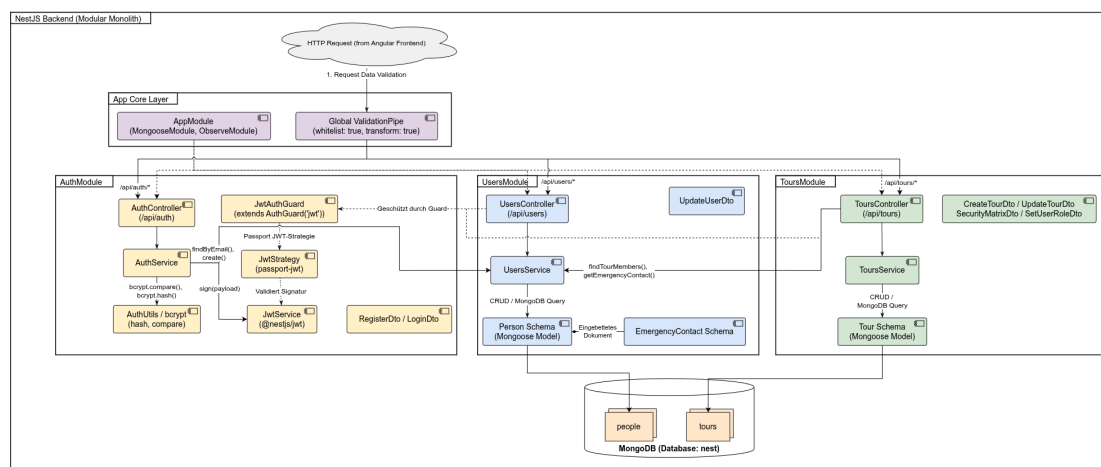


Figure 4: Bausteinsicht Level 2: NestJS Modular Monolith Struktur (PlantUML: 04_backend_architektur.puml)

- **AppModule:** Wurzelmodul, initialisiert MongoDB/Mongoose, globalen ValidationPipe und Telemetrie.
- **AuthModule:** Login, Registrierung, Passwort-Hashing (bcrypt) sowie Token-Erstellung via @nestjs/jwt und Absicherung via JwtAuthGuard (@nestjs/passport).
- **UsersModule:** Profilverwaltung; blendet sensible Notfallkontakte bei Standardabfragen immer aus (`.select('-emergencyContact')`).
- **ToursModule:** Touren, 3x3-Sicherheitsmatrix, Rollen (admin/participant) und autorisierte Notfallkontakt-Abfrage.

5 Laufzeitsicht

5.1 Szenario 1: Registrierung & Login

Ablauf bei der Registrierung eines neuen Benutzers mit Notfallkontakt:

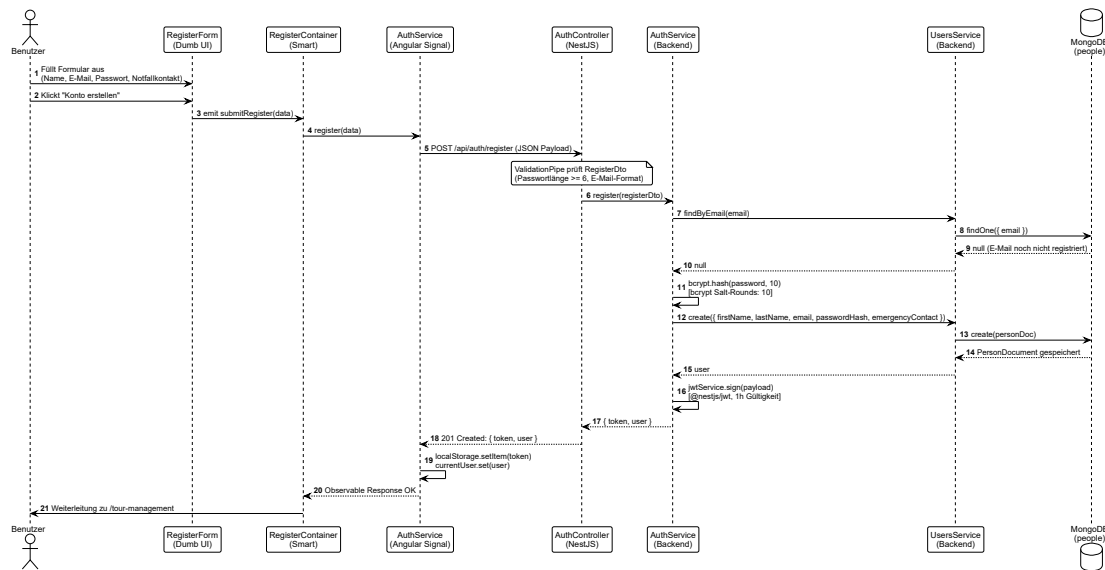


Figure 5: Laufzeitsicht: Registrierung und Token-Erstellung (PlantUML: 05_laufzeitsicht_auth.puml)

1. Benutzer füllt Formular aus (Name, E-Mail, Passwort, Notfallkontakt) und klickt "Konto erstellen".
2. ValidationPipe prüft das RegisterDto (E-Mail-Format, Passwort ≥ 6 Zeichen).
3. Backend hasht das Passwort mit Salt via bcrypt (10 Salt-Rounds) und speichert die Person in MongoDB.
4. Backend signiert ein JWT via JwtService (@nestjs/jwt, 1 Stunde gültig) und schickt es zurück.
5. Frontend speichert das Token im localStorage und leitet zur Tourenübersicht weiter.

5.2 Szenario 2: Tour erstellen mit 3x3 Sicherheitsmatrix

Ablauf beim Planen und Abspeichern einer Tour:

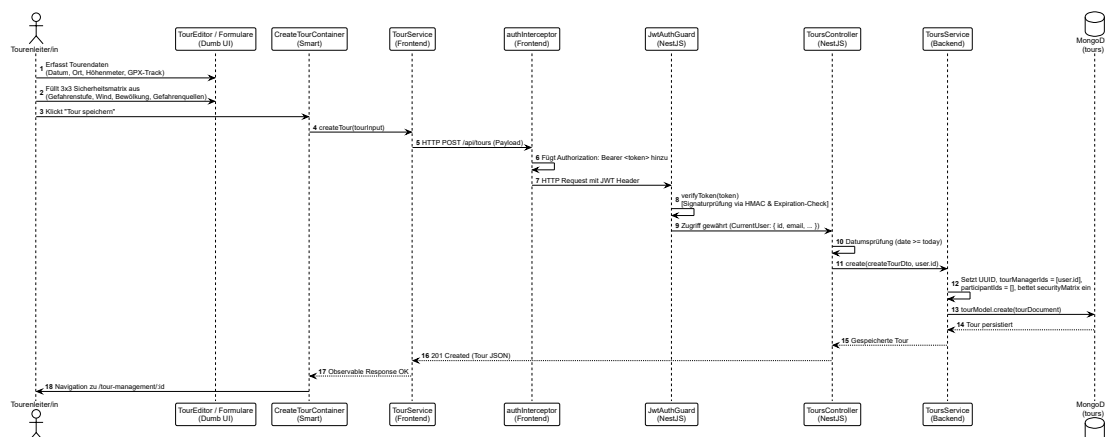


Figure 6: Laufzeitsicht: Tourenerstellung inkl. 3x3-Matrix (PlantUML: 06_laufzeitsicht_tour_erstellung.puml)

1. Tourenleiter erfasst Tourdaten, GPX-Track und füllt die 3x3-Matrix (Lawinenstufe, Wind, Gefahren) aus.

2. authInterceptor hängt das JWT als Bearer-Token an den POST-Request an /api/tours.
3. JwtAuthGuard verifiziert das Token und stellt den User bereit.
4. Controller prüft, dass das Datum nicht in der Vergangenheit liegt (date >= today).
5. Service trägt den User in tourManagerIds ein und speichert die Tour mit der Matrix in MongoDB.
6. Frontend leitet direkt auf die neue Detailansicht weiter.

5.3 Szenario 3: Tour-Beitritt & Notfallkontakt-Abfrage

Ablauf für Beitritt und Berechtigungsprüfung im Notfall:

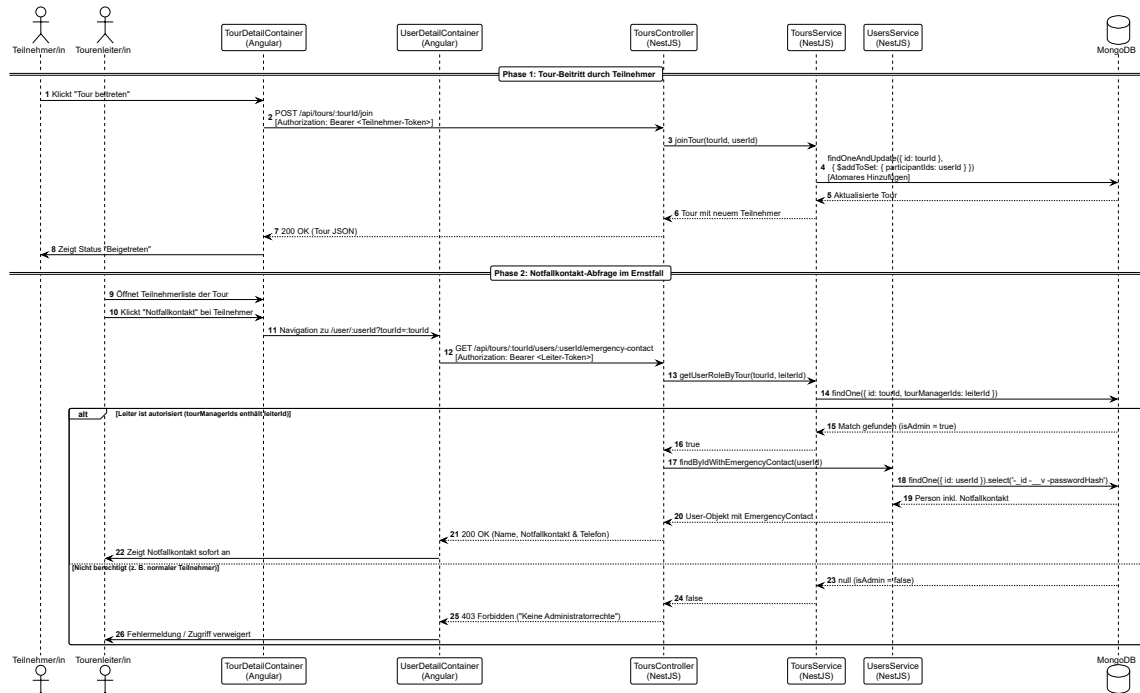


Figure 7: Laufzeitsicht: Tour-Beitritt und Berechtigungsprüfung (PlantUML: 07_laufzeitsicht_notfallkontakt.puml)

1. **Beitritt:** Teilnehmer klickt "Tour beitreten". Backend fügt seine ID atomar via \$addToSet in participantIds ein.
2. **Notfallkontakt-Abruf:** Klickt der Tourenleiter auf einen Notfallkontakt, prüft das Backend mit getUserRoleByTour():
 - **Ist Tourenleiter:** Gibt Notfallkontakt mit HTTP 200 zurück.
 - **Kein Tourenleiter:** Bricht sofort ab mit HTTP 403 Forbidden.

6 Verteilungssicht & Deployment

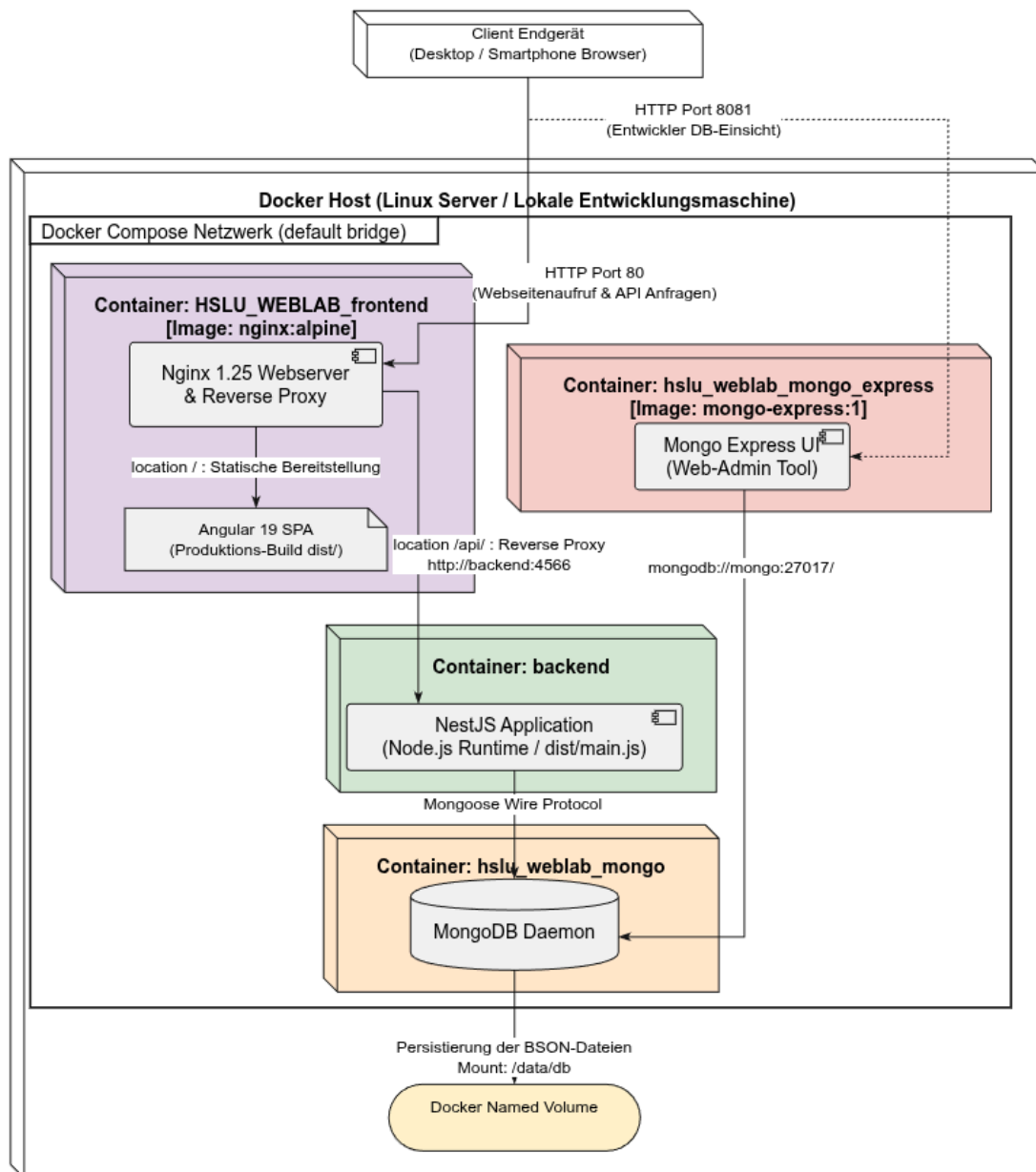


Figure 8: Verteilungssicht: Docker Compose Umgebung (PlantUML:
08_verteilungssicht_deployment.puml)

6.1 Container & Multi-Stage Builds

- **Frontend:** Stage 1 (node:24-slim) baut Angular (npm run build). Stage 2 (nginx:alpine) übernimmt nur den fertigen dist/-Ordner.
- **Backend:** Stage 1 kompiliert TypeScript. Stage 2 führt nur dist/ mit Produktions-Dependencies aus.
- **Nginx-Routing:** Serviert Angular per HTML5-Fallback (try_files \$uri /index.html) und leitet /api/ an das Backend weiter (proxy_pass http://backend:4566). Kein CORS nötig.

6.2 Automatisierte CI/CD Pipeline (GitHub Actions)

Neben dem reproduzierbaren 1-Befehl-Start via `docker compose up --build` wird die Codebasis bei jedem Push und Pull Request auf main über eine dreistufige GitHub Actions Pipeline (.github/workflows/ci.yml) automatisiert geprüft:

- **Backend-Job:** Linter (oxlint/ESLint), TypeScript-Build, 51 Unit-Tests und 20 API-Integrationstests gegen MongoMemoryServer.
- **Frontend-Job:** Dependency-Installation (npm ci), Multi-Stage Build und 121 Frontend-Unit-Tests via Vitest.
- **E2E-Job:** Cypress-Integrationstests zur Validierung der Gesamtabläufe (Authentifizierung, Tourenverwaltung, Navigation).

7 Querschnittskonzepte

7.1 Ownership, Datenschutz & Sicherheit

- **Datenschutz:** Standardabfragen filtern Notfallkontakte immer serverseitig aus (`.select(' - emergencyContact')`).
- **Authentifizierung:** Zustandsloses JWT, signiert via `@nestjs/jwt` (1h Lebensdauer) und validiert über eine Passport-JWT-Strategie (`JwtStrategy`). Passwörter werden sicher mit `bcrypt` gehasht (10 Salt-Rounds) und verifiziert.
- **Validierung & NoSQL-Schutz:** Der globale `ValidationPipe` (`whitelist: true, transform: true`) verwirft unbekannte Payload-Felder. Falsche Datumsformate oder NoSQL-Injection-Versuche (z. B. `$gt`, `$where`) werden sofort mit HTTP 400 geblockt, bevor sie die Datenbank erreichen.
- **Fehlerformat (Error Handling):** NestJS liefert Fehler standardisiert als JSON: `{"statusCode": 400, "message": [...], "error": "Bad Request"}`. Das Frontend wertet `error.message` aus und zeigt sie im UI an. Bei HTTP 401 loggt der `authInterceptor` automatisch aus.

7.2 Datenmodell & Konsistenz

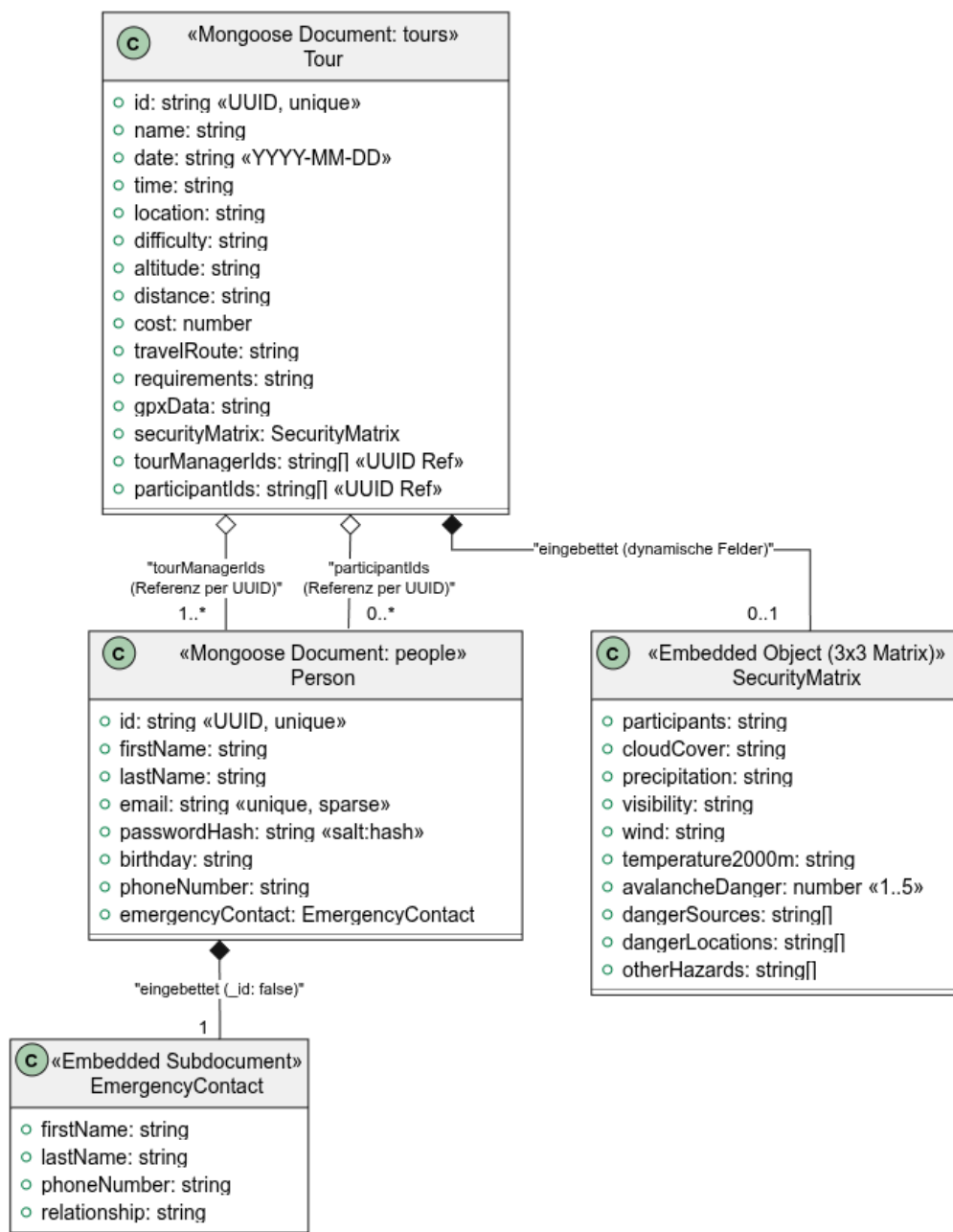


Figure 9: Datenmodell: Mongoose Schemas und Beziehungen (PlantUML: 09_datenmodell_persistenz.puml)

- **Eingebettet:** EmergencyContact in Person (`_id: false`) und SecurityMatrix in Tour.
- **UUIDs statt MongoDB ObjectIds:** Aus Gewohnheit aus früheren Projekten habe ich IDs manuell als UUIDs (`crypto.randomUUID()`) generiert und Referenzen im Service-Layer per `findByIds({ id: { $in: ids } })` statt via `Mongoose-populate()` aufgelöst. Da dies stabil lief und die Domäne sauber von Datenbank-Interna entkoppelt, habe ich es so beibehalten.

7.3 Frontend-Konzepte & Reaktivität

Thema	Umsetzung & Ansatz
Reaktivität	Angular Signals (<code>signal</code> , <code>computed</code>) für lokalen State; <code>httpResource</code> für deklaratives Laden.

Komponenten-Kommunikation	Smart Container verwalten State; Dumb Components nutzen ausschliesslich @Input und @Output.
Change Detection	ChangeDetectionStrategy.OnPush auf allen Komponenten für gute Performance.
Styling & Feedback	CSS Design-System mit Farbcodes für Lawinenwarnstufen; Validierungsfeedback via InputFieldError.
Responsive Design	Mobile-First via CSS Flexbox & Grid; grossflächige Touch-Bedienung für den Bergsport; dynamisches Leaflet-Resize. Nachgewiesen durch Lighthouse Mobile Score > 90.
Offline-Strategie	Tokens/User im localStorage. Für Notfallkontakte ohne Netz ist der Ausbau zur PWA konzipiert.

7.4 Bewusste Architektur-Abweichungen & Trade-offs

Zur Erfüllung der Qualitätsziele und Vermeidung unnötiger Komplexität wurden bewusste architektonische Abweichungen von gängigen Standardmustern gewählt:

- **Signals statt externem State-Management (NgRx/Redux):** Für den Scope der Applikation bieten Angular Signals (signal, computed, httpResource) eine leichtgewichtige, performante Reaktivität ohne komplexen Action/Reducer-Boilerplate.
- **UUIDs statt MongoDB ObjectIds & Service-Layer Resolving:** Domänen-IDs werden als UUIDs generiert. Referenzen (tourManagerIds, participantIds) werden im Service über \$inQueries aufgelöst statt über Mongoose-populate(), was die Domäne sauber von Datenbank-Internen entkoppelt.
- **Eingebettete BSON-Dokumente statt Tabellen-Normalisierung:** Die 3x3-Sicherheitsmatrix und Notfallkontakte sind als Subdokumente direkt eingebettet. Dies verhindert teure Joins und garantiert atomare Konsistenz beim Speichern.

7.5 Testing

Test-Typ	Werkzeug	Anzahl	Was wird abgedeckt?
Backend Unit	Vitest	9 Files	Services (users, tours, auth), JwtStrategy, JwtAuthGuard, DTO-Validierung, Hash-Utils.
Backend Integration	Vitest / Supertest	1 File	REST-Endpunkte gegen die Test-Datenbank (api.integration.spec.ts).
Frontend Unit	Vitest	23 Files	Smart Container, Dumb Components (security-matrix), Services, Pipes, UI-Kit.
Frontend E2E	Cypress	5 Files	Auth-Flow, Tourenverwaltung, Toureneditor, Tourdetails, Navigation.

8 Architekturentscheidungen (ADRs)

Architekturentscheidungen dokumentieren wesentliche Technologie- und Strukturwahlen inklusive des Entscheidungskontexts, der verworfenen Alternativen und der resultierenden Trade-offs:

8.1 ADR-01: Backend mit NestJS (Modularer Monolith)

- **Kontext & Problem:** Gesucht war ein robustes Backend-Framework mit klarer Schichtenarchitektur, Dependency Injection und automatischer Request-Validierung.
- **Entscheidung:** NestJS 11 auf Node.js 24 mit TypeScript. Modularer Monolith unterteilt in fachliche Domänen (auth, tours, users). Validierung über DTOs mit class-validator.
- **Verworfen Alternativen:** **Express.js (barebone):** Bietet keine strukturierte Schichtenarchitektur und keinen DI-Container, was bei wachsender Domänenlogik rasch zu unübersichtlichem Code führt. **Microservices:** Zu hoher Deployment-, Netzwerk- und Monitoring-Overhead für das Projekt.
- **Konsequenzen & Trade-offs:** (+) Hohe Typsicherheit, klare Schichtentrennung (Controller, Service, Modul), identische Denkweise wie in Angular. (-) Einarbeitungsaufwand in TypeScript-Decorators und NestJS-Modul-Wiring.

8.2 ADR-02: Frontend mit Angular 19 Standalone & Signals

- **Kontext & Problem:** Aufbau einer reaktiven Single Page Application mit schnellen Ladezeiten, klarer Trennung von UI und State sowie minimalem Boilerplate.
- **Entscheidung:** Angular 19 mit Standalone Components, Angular Signals (signal, computed, httpResource) und Smart/Dumb-Muster.
- **Verworfen Alternativen:** **React:** Kein integriertes Routing- oder Forms-System; erfordert viele heterogene Dritt-Bibliotheken. **Angular mit NgRx:** Hohe Boilerplate-Menge (Actions, Reducers, Effects, Selectors), die für die Projektgrösse unverhältnismässig gewesen wäre.
- **Konsequenzen & Trade-offs:** (+) Keine veralteten NgModules, feingranulare Reaktivität ohne RxJS-Subscription-Leaks, optimale Performance durch OnPush. (-) Sehr neue APIs (httpResource), wenig historische Dokumentation.

8.3 ADR-03: Persistenz mit MongoDB & Mongoose (Embedded Aggregates)

- **Kontext & Problem:** Speicherung von Touren mit hierarchischen, dynamischen 3x3-Matrixfeldern und Notfallkontakten.
- **Entscheidung:** MongoDB 8.0 mit Mongoose ODM. Die 3x3-Sicherheitsmatrix und Notfallkontakte werden direkt als eingebettete BSON-Subdokumente abgelegt.
- **Verworfen Alternativen:** **Relationale SQL-Datenbank (PostgreSQL/MySQL):** Hätte für die 3x3-Matrix (Hangneigungen, Gefahrenquellen, Wetterzonen) 3–4 Join-Tabellen erfordert und bei späteren Schema-Anpassungen komplexe DDL-Migrationen bedingt.
- **Konsequenzen & Trade-offs:** (+) Schnelle Lese- und Schreibzugriffe ohne Joins; atomare Updates via \$addToSet; flexible Erweiterbarkeit. (-) Keine transaktionalen Fremdschlüssel auf Datenbankebene; relationale Integrität (z. B. User-IDs in Touren) muss im Service-Layer validiert werden.

8.4 ADR-04: Zustandsloses JWT mit Passport & bcrypt

- **Kontext & Problem:** Absicherung der REST-Endpunkte ohne serverseitigen Session-State, um Skalierbarkeit und einfache Containerisierung zu gewährleisten.
- **Entscheidung:** Zustandsloses JWT mit 1 Stunde Lebensdauer, signiert via @nestjs/jwt und validiert per Passport-Strategie (JwtStrategy). Passwörter werden mit bcrypt (10 Salt-Rounds) gehasht.

- **Verworfenen Alternativen: Stateful Sessions mit Redis-Store:** Hätte eine zusätzliche Datenbank erfordert und das Docker-Setup verkompliziert. **Eigenbau-Kryptografie:** Hohes Sicherheitsrisiko.
- **Konsequenzen & Trade-offs:** (+) Völlig zustandslos, ideal für Container; Standard im NestJS-Ökosystem. (-) Logout invalidiert Tokens nur clientseitig im localStorage (Token-Blacklist als technische Schuld dokumentiert).

8.5 ADR-05: Deployment mit Docker Compose & Nginx Reverse Proxy

- **Kontext & Problem:** Reproduzierbarer 1-Befehl-Start auf beliebigen Entwicklerrechnern ohne lokale Abhängigkeiten und Vermeidung von Cross-Origin Resource Sharing (CORS) Problemen.
- **Entscheidung:** Docker Compose Stack mit Multi-Stage Dockerfiles. Ein Nginx-Reverse-Proxy lauscht auf Port 80, serviert das kompilierte Frontend und leitet /api/* intern an das Backend weiter.
- **Konsequenzen & Trade-offs:** (+) Zuverlässiger Start mit `docker compose up --build`; vollständiges Zero-CORS-Routing. (-) Nginx-Konfiguration muss gepflegt werden.

8.6 ADR-06: Restriktiver Notfallkontakt-Zugriff (Least Privilege)

- **Kontext & Problem:** Notfallkontakte sind hochsensible Personendaten. Unbefugte Teilnehmende dürfen keinesfalls Einsicht in private Notfallnummern anderer Gruppenmitglieder erhalten.
- **Entscheidung:** Serverseitige Filterung per Default (`.select('-emergencyContact')`). Abruf ausschließlich über den Endpunkt `/api/tours/:tourId/users/:userId/emergency-contact`, geschützt durch Prüfung der Leitungsrolle (`tourManagerIds`).
- **Verworfenen Alternativen: Clientseitiges Ausblenden:** Fatale Sicherheitslücke, da die Notfallkontakte im Browser-Netzwerk-Tab für alle lesbar gewesen wären.
- **Konsequenzen & Trade-offs:** (+) Garantierter Datenschutz nach DSGVO/DSG (**Principle of Least Privilege**). (-) Erfordert einen separaten HTTP-Request im Ernstfall (durch Qualitätsszenario Q2 mit Ladezeit < 200 ms abgesichert).

9 Qualitätsanforderungen

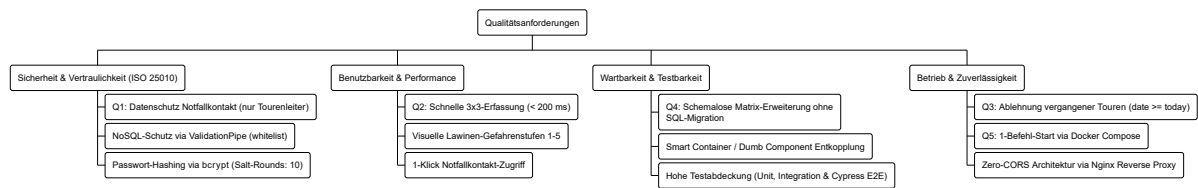


Figure 10: Qualitätsbaum: Gliederung der Qualitätsziele nach ISO 25010 (PlantUML: 10_qualitaetsbaum.puml)

ID	Szenario	Verhalten des Systems	Messkriterium
Q1	Teilnehmer ruft Notfall-URL eines anderen Teilnehmers auf.	Backend prüft Leiterrechte und blockiert den Zugriff.	HTTP 403 Forbidden.
Q2	Tourenleiter speichert 3x3-Sicherheitsmatrix ab.	Formular validiert Daten; Matrix wird sofort gespeichert und gerendert.	Ladezeit < 200 ms lokal.
Q3	Beitrittsversuch zu einer vergangenen Tour.	Backend prüft <code>date < today</code> und lehnt den Beitritt ab.	HTTP 400 Bad Request.
Q4	Matrix wird später um neue Gefahrenfelder erweitert.	Da MongoDB schemalos speichert, muss nur das DTO angepasst werden.	Keine SQL-Migration nötig.
Q5	Start auf einem frischen Entwicklungsrechner mit Docker.	Startet Frontend, Backend und Datenbank ohne manuelle Vorarbeit.	<code>docker compose up</code> auf Port 80.
Q6	Bewertung der Web-Performance und Qualität mit Google Lighthouse (Mobile & Desktop).	Die Anwendung lädt effizient, ist barrierefrei, suchmaschinenoptimiert und erfüllt Web-Best-Practices.	Scores für Performance, Accessibility, Best Practices und SEO betragen jeweils > 90.

9.1 Lighthouse

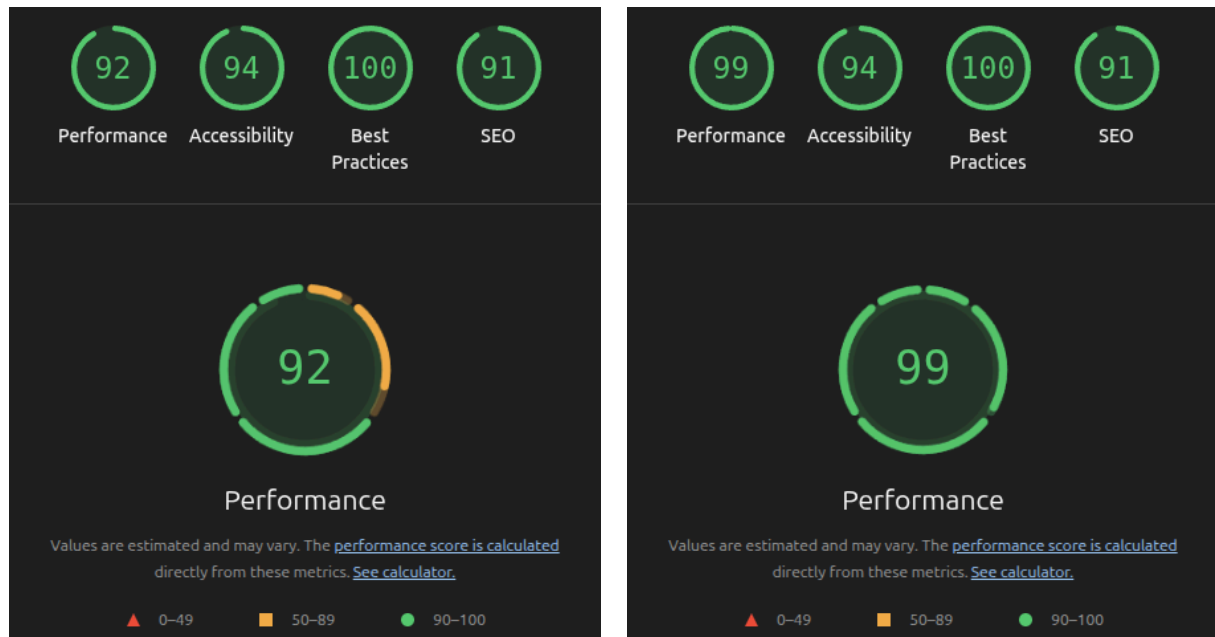


Figure 11: Lighthouse-Reports: Mobile (links) und Desktop (rechts) zeigen in allen Qualitätsmetriken Scores von über 90.

10 Risiken & technische Schulden

Bereich	Problem / Risiko	Status & Massnahme
Offline-Nutzung am Berg	Im alpinen Funkloch schlägt das Neuladen fehl; Notfallkontakte wären offline nicht erreichbar.	Konzipiert als nächster Schritt: PWA mit Service Worker und IndexedDB-Cache für Notfallnummern.
GPX-Parsing im Browser	Riesige GPX-Dateien (> 20 MB) könnten den UI-Thread kurz blockieren.	Aktuell für normale GPX-Tracks ausreichend; künftig Auslagerung in einen Web Worker.
E2E-Testisolation	Cypress-Tests teilen sich teilweise Datenbankzustände.	Tests bereinigen ihre Daten eigenständig; für Einzelentwickler im Modul WebLab vollkommen ausreichend.